

Multi-Store Matching Engine

Score = distance + price factor + rating + delivery time + surge penalty

Score = $w_1D + w_2P + w_3R + w_4T + w_5S$

```

public class StoreMatchingService {
    @Autowired private StoreRepository storeRepo;
    @Autowired private SurgeService surgeService;

    public List<Store> findBestStores(double lat, double lon) {
        List<Store> stores = storeRepo.findNearbyCandidates(lat, lon);
        List<ScoredStore> scored = new ArrayList<>();

        for (Store s : stores) {
            double distance = haversine(lat, lon, s.getLat(), s.getLon());
            double surgePrice = surgeService.getSurgePrice(s.getId());

            // Lower scores indicate better stores:
            // distance (50%), rating preference (20%), delivery time (20%),
            // and surge pricing (10%) are combined into a single ranking score.
            // Surge price means an extra price applied during high-demand periods
            // For example, imagine a food-delivery system:
            // - Normal delivery fee: ₹40
            // - High demand: ₹60
            // - Very high demand: ₹80
            double score =
                distance * 0.5 +
                (1 / s.getRating()) * 0.2 +
                s.getBaseDeliveryTime() * 0.2 +
                surge * 0.1;

            scored.add(new ScoredStore(s, score));
        }

        return scored.stream()
            .sorted(Comparator.comparingDouble(ScoredStore::score))
            .map(ScoredStore::store)
            .toList();
    }
}

```

One important note: $(1 / s.getRating()) * 0.2$ means a higher rating produces a lower score, which is consistent with the comment if lower scores are better. However, the units of distance, deliveryTime, and surge are very different, so in a real scoring model you'd usually **normalize these values before applying the weights.**

is effectively saying: *Prefer stores that are closer, have better ratings, deliver faster, and have less surge pricing.*

1. The Multi-Store Problem

A food-delivery platform may have:

Customer	Each store can have different:	
	distance	reliability
+---- Store A	inventory	pricing
+---- Store B	preparation time	rating
+---- Store C	operating hours	traffic conditions
+---- Store D	delivery zone	driver availability
+---- Store E	capacity	

Therefore: Nearest Store does not necessarily mean: Best Store

2. What the Matching Engine Must Decide

The engine may need to answer:

Which store?

Can the store fulfill the entire cart?

How long will preparation take?

How long will delivery take?

Can the store deliver to the customer?

Is the store accepting orders?

Does the store have enough inventory?

Does the store have capacity?

Are drivers available?

What will the customer pay?

What is the expected delivery time?

Is splitting the order better?

2.1. Store Availability

ACTIVE

INACTIVE

BUSY

TEMPORARILY_CLOSED

CLOSED

2.2. Store Operating Hours

The matching engine should
therefore consider:

currentTime

storeOpeningTime

storeClosingTime

2.3. Store Availability

Store exists + Store is active +

Store accepts orders +

Store is currently open

2.4. Store Inventory

Suppose the customer requests:

Chicken Biryani × 2

Store A has: quantity = 1

Store B has: quantity = 5

Store A cannot fully satisfy the order.

**Therefore: Inventory Validation must happen
before final ranking.**

2.5. Store Preparation Time

Store A → 15 min

Store B → 35 min

Store C → 10 min

Preparation time directly affects:

2.6. Store Geographic Location

Every store should maintain:

latitude

longitude

2.7. Redis GEO for Store Discovery

Redis GEO can efficiently find nearby stores.

For example:

GEOADD stores 78.4867 17.3850 store-A

GEOADD stores 78.4900 17.3900 store-B

GEOADD stores 78.4800 17.3800 store-C

Then:

GEOSEARCH can retrieve nearby stores.

2.8. Candidate Search Radius

A configurable radius can be used:

```
initialRadius = 3 km;
```

If insufficient candidates exist:

```
3 km → 5 km → 8 km → 10 km
```

For example:

```
Search 3 km  
→ 1 store
```

```
Search 5 km  
→ 4 stores
```

```
Stop
```

There is no reason to search 20 km if enough eligible candidates are already available.

2.9. Expanding the Search Radius

Use progressive expansion:

```
radius = 3 km
```

```
while candidates < minimumCandidates:  
    radius = radius * 2  
    search(radius)
```

Example:

```
3 km  
6 km  
12 km
```

The exact strategy should be configurable.

3. Store Eligibility Filtering

After candidate generation:

```
Candidate Stores
|
v
Eligibility Filter
```

Remove stores that fail hard constraints.

Example:

```
Store A → inactive      → REMOVE
Store B → no inventory   → REMOVE
Store C → closed         → REMOVE
Store D → eligible       → KEEP
Store E → eligible       → KEEP
```

4. Hard Constraints vs Soft Constraints

Hard Constraints:

A candidate either satisfies them or doesn't.

Examples:

```
Store is open
Store accepts orders
Store delivers to customer
Inventory available
Store active
```

If false:

```
Candidate = INVALID
```

Soft Constraints:

These affect ranking.

Examples:

```
distance
ETA
rating
preparation time
reliability
delivery cost
```

A store can be worse in one category but better overall.

5. Driver Availability

Store matching can also depend on driver availability.

Suppose:

```
Store A
Nearby drivers = 0

Store B
Nearby drivers = 5
```

Even if Store A is closer, Store B may produce a faster delivery.

Therefore:

Store Matching and **Driver Matching** should eventually work together.

6. Store Capacity

Stores can become overloaded.

Example:

Store A

Current active orders = 45
Maximum capacity = 50

Store A still technically accepts orders, but adding another order may increase preparation time significantly.

Therefore:

Capacity utilization can become a scoring feature.

Example

utilization =
activeOrders / maxCapacity

7. Inventory Validation

Inventory should be checked against the entire cart.

Example:

Cart:

Burger × 2
Coke × 1
Fries × 1

Store:

Burger = 2
Coke = 10
Fries = 0

Result:

FULL_FULFILLMENT = false

8. Store Preparation Time

Preparation time can be static or dynamic

Preparation Time = base preparation time + current queue delay + order complexity

9. Candidate Feature Vector

distance = 2.4 km
roadDistance = 3.1 km
preparationTime = 12 min
driverETA = 6 min
storeRating = 4.6
reliability = 0.95
capacityUtilization = 0.40
inventoryCompleteness = 1.0
deliveryCost = ₹35

These features become the input to the ranking system.

10. Store Scoring

A simple scoring model could be:

score =
distanceScore
+ etaScore
+ preparationScore
+ reliabilityScore
+ ratingScore
+ costScore

Each component should be normalized. Otherwise: distance = 3.2 and rating = 4.8 would not be directly comparable.

11. Weighted Scoring Model

For example:

```
score =  
  0.30 × ETA  
+ 0.20 × preparation  
+ 0.15 × distance  
+ 0.15 × reliability  
+ 0.10 × rating  
+ 0.10 × cost
```

The actual weights should be configurable.
StoreMatchingWeights {
 etaWeight;
 preparationWeight;
 distanceWeight;
 reliabilityWeight;
 ratingWeight;
 costWeight;
}

12. Customer Experience Score

The system may eventually calculate `customerExperienceScore` based on:

ETA
delivery fee
store rating
historical reliability
item availability

For example: CX Score =
0.40 × ETA
+
0.20 × reliability
+
0.15 × rating
+
0.15 × availability
+
0.10 × cost

13. Store Reliability Score

Historical performance can be used.

Example:

Store A:

On-time rate = 98%
Cancellation rate = 1%
Average preparation delay = 2 min

versus:

Store B

On-time rate = 75%
Cancellation rate = 8%
Average preparation delay = 12 min

Store A may be preferable even if Store B is slightly closer.

14. The Best Store Is Not Always the Nearest Store

Historical performance can be used.

```
bestStore = argmax(  
  eligibility, inventory, ETA, preparation,  
  reliability, cost, capacity, customerExperience  
)
```

15. Fulfillment Strategy

The engine should optimize:

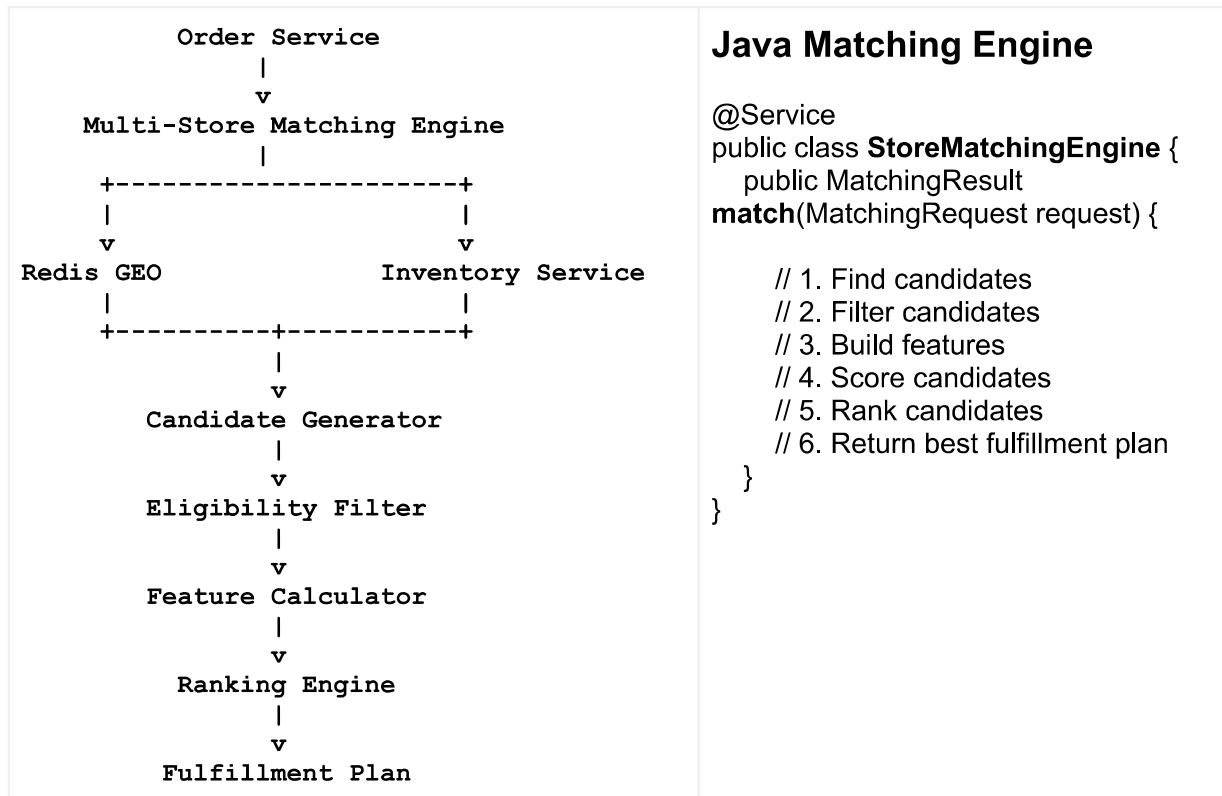
number of stores + total preparation time + total travel time + delivery cost + customer ETA

Example:

Plan A	Plan B
Store A 100% cart	Store A = 60% Store B = 40%
ETA = 35 min Cost = ₹40	ETA = 50 min Cost = ₹80

Plan A wins.

16. Production Matching Pipeline



17. Store Candidate Service

Responsible for: **Finding geographically relevant stores**

```
public interface StoreCandidateService {  
    List<StoreCandidate> findCandidates(  
        double latitude,  
        double longitude,  
        double radiusKm  
    );  
}
```

Implementation:
Redis GEO

18. Store Candidate Service

Responsible for hard constraints.

```
public interface StoreEligibilityService {  
    boolean isEligible(  
        StoreCandidate store,  
        MatchingRequest request  
    );  
}
```

Checks:
open?
active?
accepting orders?
delivery zone?
inventory?
capacity?

19. Store Candidate Service

Responsible only for ranking.

```
public interface StoreScoringService {  
    double score(StoreFeatures features);  
}
```

This allows different implementations:

RuleBasedStoreScoringService
MLStoreScoringService

20. Matching Engine Orchestrator

The orchestrator coordinates everything:

```
MatchingEngine  
|  
+-- CandidateService  
|  
+-- EligibilityService  
|  
+-- InventoryService  
|  
+-- ETAService  
|  
+-- PricingService  
|  
+-- ScoringService  
|  
+-- FulfillmentPlanner
```

21. Example Matching Algorithm

```
match(order) {
  candidates = findNearbyStores(order.customerLocation);
  eligible = filterEligible(candidates, order);

  for each store {
    inventory = validateInventory(store, order);

    if (inventory is invalid) continue;
    features = buildFeatures(store, order);
    score = calculateScore(features);
    rankedCandidates.add(store, score);
  }

  sort candidates by score
  bestSingleStore = highest ranked candidate

  if(bestSingleStore fully satisfies order):
    return bestSingleStore
  else:
    generate multi-store plans
    evaluate plans
    return best plan
}
```

Matching Request	Matching Response	For multi-store Response:
<pre>public class MatchingRequest { UUID orderId; UUID customerId; double customerLatitude; double customerLongitude; List<OrderItem> items; DeliveryPriority priority; }</pre>	<pre>public class MatchingResult { UUID storeId; List<OrderItem> fulfilledItems; double score; double distanceKm; int preparationTimeMinutes; int estimatedDeliveryMinutes; BigDecimal deliveryFee; }</pre>	<pre>public class FulfillmentPlan{ List<StoreFulfillment> fulfillments; int totalEta; BigDecimal totalCost; double score; }</pre>

22. Example Java Implementation

```
@Service
@RequiredArgsConstructor
public class StoreMatchingEngine {
    private final StoreCandidateService candidateService;
    private final StoreEligibilityService eligibilityService;
    private final InventoryService inventoryService;
    private final StoreFeatureService featureService;
    private final StoreScoringService scoringService;

    public MatchingResult match(MatchingRequest request) {
        List<StoreCandidate> candidates =
            candidateService.findCandidates(
                request.customerLatitude(),
                request.customerLongitude(),
                5.0
            );

        List<ScoredStore> scoredStores = new ArrayList<>();

        for (StoreCandidate candidate : candidates) {
            if (!eligibilityService.isEligible(candidate, request)) {
                continue;
            }

            InventoryResult inventory =
                inventoryService.check(candidate.storeId(), request.items());

            if (!inventory.isFullFulfillment()) {
                continue;
            }

            StoreFeatures features = featureService.build(candidate, request, inventory);
            double score = scoringService.score(features);
            scoredStores.add(new ScoredStore(candidate, score));
        }

        return scoredStores.stream()
            .max(Comparator.comparingDouble(ScoredStore::score))
            .map(this::toResult)
            .orElseThrow(
                () -> new NoStoreAvailableException(
                    "No eligible store found"
                )
            );
    }
}
```

23. Store Events

Useful events include:

```
STORE_OPENED
STORE_CLOSED
STORE_BUSY
STORE_AVAILABLE
STORE_INVENTORY_CHANGED
STORE_CAPACITY_CHANGED
STORE_PREPARATION_TIME_CHANGED
STORE_ORDER_ACCEPTED
STORE_ORDER_REJECTED
```

24. Matching Failure Handling

Matching can fail because:

```
No nearby stores
No store has inventory
All stores closed
All stores overloaded
No delivery zone available
Inventory changed during matching
```